VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5 Max Marks: 100

Student Name Vedant Sunil Patil

Registration Number 24BAI10122

Slot A24

Date of Submission 31-03-2026

1. Introduction

When I first dipped my toes into the world of programming, I honestly didn't see the point of "version control." It sounded like an enterprise-level buzzword that had nothing to do with me. My personal system was as primitive as it gets I would manually copy a folder, rename it `Project_Backup_March_10`, and hope for the best. This "copy-paste" method worked for small, single-file scripts, but the moment I started working on anything with multiple modules or dependencies, it became a nightmare. I found myself staring at five different versions of the same file, having no clue which one actually contained the working code.

That frustration is what eventually pushed me toward Git. In the beginning, I'll admit I used it like a basic cloud storage just a sequence of `git add`, `git commit`, and `git push` without really understanding the why behind the commands. However, during the course of this project, I took a deep dive into the internal logic of Git. I've realized it isn't just a tool for uploading code; it's a sophisticated system designed for structural integrity and collaboration.

Unlike older systems, Git is Distributed, meaning every single contributor has a full-fledged copy of the project history on their machine. This realization changed my perspective; it means Git is built for resilience. It's a technical manifestation of the open-source philosophy, where the power is shared among the community rather than locked in a central server.

Part A — Origin and Philosophy

A1. The Problem Git Was Created to Solve

Git was created in 2005 by Linus Torvalds. At that time, the Linux kernel project relied on a proprietary system called BitKeeper. Due to licensing conflicts, access to this tool was lost.

This situation created a serious issue. The Linux kernel is developed by contributors across the world, and without a proper version control system, managing changes becomes extremely difficult.

Instead of switching to another proprietary tool, Linus Torvalds decided to build a new system from scratch. His goal was to design something that was fast, reliable, and suitable for large-scale distributed development.

Git was built with a few important principles:

- Speed in handling operations
- Distributed architecture
- Strong data integrity
- Flexibility in workflows

From my understanding, Git was not created as a commercial product but as a solution to a real-world problem. This reflects how many open source tools originate from practical needs.

A2. The License — What It Actually Says

Git is released under the GNU General Public License (GPL v2). This is a copyleft license that ensures the software remains free and open.

The four freedoms of free software are:

1. Freedom to run the program
2. Freedom to study its source code
3. Freedom to modify it
4. Freedom to distribute copies

Git satisfies all of these freedoms.

An important aspect of GPL is that modified versions must also remain open-source. This prevents companies from taking the code and making it proprietary.

In contrast, permissive licenses like MIT allow more freedom for commercial use but do not enforce openness. In my opinion, GPL promotes fairness and long-term openness, while MIT supports flexibility and wider adoption.

A3. The Ethics of Open Source

Open-source software is not just about code but also about values.

One important question is whether all software should be open-source. While open source encourages transparency and collaboration, companies may need to protect their work for business reasons.

Another issue is whether companies should profit from open-source tools. Many organizations use tools like Git and Linux to build products. I believe this is acceptable if they contribute back to the community.

The idea of “standing on the shoulders of giants” becomes clear here. Git was created to support Linux development, and today it supports millions of projects globally.

From my perspective, open source does not reduce innovation. Instead, it creates a foundation that allows innovation to grow faster.

Part B — Linux Footprint (Unit 2)

To study how Git exists within a Linux system, I used Ubuntu through Windows Subsystem for Linux (WSL). All shell scripting tasks were performed in this environment.

In this section, I focused on understanding how Git is typically installed, managed, and structured within Linux systems.

Installation Method

Git is commonly installed using package managers such as `apt` in Ubuntu.

sudo apt update

sudo apt install git

This installs Git from official repositories, ensuring compatibility and security.

```
root@LAPTOP-BIOOQKCV: ~
root@LAPTOP-BIOOQKCV:~# sudo apt update
sudo apt install git
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1551 kB]
Get:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [250 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadata [10.7 kB]
Get:9 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Packages [1166 kB]
Get:10 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1847 kB]
Get:11 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [74.2 kB]
Get:12 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:13 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [212 B]
Get:14 http://archive.ubuntu.com/ubuntu noble-updates/main Translation-en [339 kB]
Get:15 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Components [177 kB]
Get:16 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 c-n-f Metadata [16.9 kB]
Get:17 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Packages [1665 kB]
Get:18 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 Components [386 kB]
Get:19 http://archive.ubuntu.com/ubuntu noble-updates/restricted amd64 Components [212 B]
Get:20 http://archive.ubuntu.com/ubuntu noble-updates/multiverse amd64 Components [940 B]
Get:21 http://archive.ubuntu.com/ubuntu noble-backports/main amd64 Components [7364 B]
Get:22 http://archive.ubuntu.com/ubuntu noble-backports/universe amd64 Components [13.2 kB]
Get:23 http://archive.ubuntu.com/ubuntu noble-backports/restricted amd64 Components [216 B]
Get:24 http://archive.ubuntu.com/ubuntu noble-backports/multiverse amd64 Components [212 B]
Fetched 7905 kB in 8s (998 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
```

Key Directories

Typical locations include:

- `/usr/bin/git` — executable
- `/etc/gitconfig` — system configuration
- `~/.gitconfig` — user configuration

Each repository also contains a `.git` folder that stores internal data.

User and Permissions

Git runs under the current user. It uses Linux file permissions to control access. This ensures that only authorized users can modify repositories.

Service Management

Git is not a background service. It runs only when commands are executed.

Update Model

Updates are provided through package repositories:

sudo apt update

sudo apt upgrade

Observation

Even though Git itself was not heavily used in practical execution, studying its integration helped me understand how open-source tools are structured in Linux.

Part C — The FOSS Ecosystem (Units 3 & 4)

When I started looking at Git in isolation, it seemed like just a version control tool. But while exploring further, I realized that Git is actually part of a much larger open-source ecosystem. It does not work alone instead, it connects with multiple tools, platforms, and communities.

One important aspect is the dependencies that Git relies on. Git uses libraries such as OpenSSL for secure communication and zlib for data compression. These dependencies are themselves open-source, which shows how different projects build on top of each other. This interconnected structure is a key characteristic of the FOSS ecosystem.

At the same time, Git has enabled the development of many other tools. Platforms like GitHub, GitLab, and Bitbucket are built around Git. These platforms provide features such as code hosting, collaboration, issue tracking, and automation pipelines. Without Git, these platforms would not exist in their current form.

What I found interesting is that Git itself does not try to do everything. It focuses only on version control, and other tools extend its functionality. This reflects the Unix philosophy of doing one thing well.

Another important part of the ecosystem is the community. Git is maintained by contributors from different parts of the world. Discussions happen through mailing lists, forums, and repositories. Even though there is no single company controlling Git, it continues to evolve through collaboration.

Git also plays a role in modern development workflows such as:

- pull requests
- branching strategies
- continuous integration

These practices are now standard in software development, and many of them depend directly on Git.

From my perspective, Git is not just a tool but a foundation that supports a large network of software and developers. It shows how open-source projects grow by connecting with each other rather than working independently.

Part D — Open Source vs Proprietary (Critical Analysis)

When comparing Git with proprietary version control systems, I realized that the difference is not only about features but also about mindset and approach.

The first difference is cost. Git is completely free and can be used without any restrictions. This makes it accessible to students, independent developers, and organizations of any size. Proprietary tools, on the other hand, often require licenses or subscriptions. While they may offer additional services, the cost can be a limitation.

Another major difference is transparency. Since Git is open source, its code can be inspected by anyone. Initially, I thought this might make it less secure, but it actually works the other way. When more people can review the code, problems can be identified and fixed quickly. Proprietary tools do not provide this level of visibility, so users have to rely on the company.

Support is another area where differences are visible. Proprietary software usually comes with dedicated customer support, which can be useful in critical situations. Git mainly relies on community support. However, in practice, I noticed that community resources are quite extensive and often provide solutions quickly.

Flexibility is one of the strongest points of Git. Developers can customize workflows, create branches, and manage projects according to their needs. Proprietary tools often have fixed workflows, which may not suit every situation.

Control is also an important factor. Git is developed and maintained by a community, which means decisions are influenced by contributors. Proprietary tools are controlled by companies, and decisions are made based on business priorities.

After comparing both, I feel that Git is more suitable for environments where flexibility and transparency are important. At the same time, proprietary tools may be useful in organizations that require structured support and controlled systems.

Overall, this comparison helped me understand that there is no single “best” option. The choice depends on the requirements, but Git provides a strong balance of power, freedom, and reliability.

Dimension	Open Source (Git)	Proprietary Alternative
Cost	• Free • No license	• Paid • Subscription needed

Security	• Code visible • Public review	• Code hidden • Trust company
Support	• Community help • No official support	• Official support • Faster response
Freedom to modify	• Fully customizable • Flexible use	• No modification • Fixed features
Community vs control	• Community-driven • Open decisions	• Company-controlled • Business decisions
Overall verdict	• Flexible and open • Good for developers	• Stable and supported • Less flexible

After comparing both open-source and proprietary tools, I realized that the difference is not just about features, but also about how they are designed and used. Git gives a lot of freedom to the user. You can explore, modify, and work in your own way without many restrictions. This makes it very useful for developers who want flexibility and control over their work.

At the same time, proprietary tools are not completely bad. They offer structured support and are easier to manage in some professional environments. But personally, I would prefer using Git because it feels more open and adaptable. It also allows learning at a deeper level since you are not limited by the system. If given a chance, I would definitely like to contribute to such open-source projects in the future.

Part E — Shell Script Tasks and Explanation

As part of this project, I created and executed five shell scripts using the Ubuntu terminal. While working on these scripts, I focused more on understanding how the logic works rather than memorizing specific commands.

Initially, I faced some confusion in writing the scripts, especially in structuring them properly. But after running them a few times and correcting small mistakes, I became more comfortable with how shell scripting works.

Script 1 — System Identity Report

This script displays basic information about the system such as the current user, system uptime, and other details.

While working on this script, I understood that the system already contains all the required information. The main task was to collect and display it in a proper format.

At first, my output was not well organized, but after making a few changes, it became clearer and easier to read. This script helped me understand how to present system information effectively.

```
root@LAPTOP-BIOOQKCV:~# rm script1.sh
root@LAPTOP-BIOOQKCV:~# nano script1.sh
root@LAPTOP-BIOOQKCV:~# chmod +x script1.sh
root@LAPTOP-BIOOQKCV:~# ./script1.sh
=====================================
      The Open Source Audit
=====================================
Student Name    : Vedant Patil
Chosen Software : Git
-------------------------------------
System Identity Report
-------------------------------------
Distribution    : Ubuntu 24.04.1 LTS
Kernel Version  : 6.6.87.2-microsoft-standard-WSL2
Logged-in User  : root
Home Directory  : /root
System Uptime   : up 25 minutes
Current Date    : Sunday, 29 March 2026 18:06:52
-------------------------------------
License Info    : This operating system (Linux) is covered under the GNU General Public License v2.0 (GPLv2).
=====================================
root@LAPTOP-BIOOQKCV:~#
```

Script 2 — FOSS Package Inspector

This script checks whether a particular software is installed and displays some basic details about it.

The main learning here was understanding how a script can make decisions based on conditions. Depending on whether the package was found or not, the script produced different outputs.

This helped me understand how Linux systems manage installed software and how that information can be accessed.

```
root@LAPTOP-BIOOQKCV:~# nano script2.sh
root@LAPTOP-BIOOQKCV:~# chmod +x script2.sh
root@LAPTOP-BIOOQKCV:~# ./script2.sh
========================================
      FOSS Package Inspector
========================================
Inspecting package: git
----------------------------------------
[SUCCESS] git is installed on this system.
----------------------------------------
Package Details:
Architecture: amd64
Version: 1:2.43.0-1ubuntu7.1
----------------------------------------
FOSS Philosophy Note:
Git: The distributed engine of collaborative freedom, built by Linus Torvalds.
========================================
```

Script 3 — Disk and Permission Auditor

This script analyzes different directories and shows their size and permissions.

While working on this, I understood how the same task can be repeated for multiple directories. Initially, it felt confusing, but after running the script, I saw how it simplifies repetitive work.

It also helped me understand how permissions work in Linux and why they are important for system security.

```
=====================================
    Directory & Permission Auditor
=====================================
[Directory] /etc
  -> Permissions & Owner: drwxr-xr-x root root
  -> Size: 4.2M
-------------------------------------
[Directory] /var/log
  -> Permissions & Owner: drwxrwxr-x root syslog
  -> Size: 8.8M
-------------------------------------
[Directory] /home
  -> Permissions & Owner: drwxr-xr-x root root
  -> Size: 4.0K
-------------------------------------
[Directory] /usr/bin
  -> Permissions & Owner: drwxr-xr-x root root
  -> Size: 92M
-------------------------------------
[Directory] /tmp
  -> Permissions & Owner: drwxrwxrwt root root
  -> Size: 40K
-------------------------------------
=====================================
      Chosen Software Audit (Git)
=====================================
[FOUND] Git configuration/binary located at: /usr/bin/git
  -> Permissions & Owner: -rwxr-xr-x root root
=====================================
root@LAPTOP-BIOOQKCV:~#
```

Script 4 — Log File Analyzer

This script reads a file and counts how many times a specific keyword appears.

This part was slightly more challenging because it required checking the file step by step. However, once it worked correctly, I understood how useful it is for analyzing large amounts of data.

It showed how automation can make tasks easier and faster.

```
root@LAPTOP-BIOOQKCV:~# nano script4.sh
root@LAPTOP-BIOOQKCV:~# chmod +x script4.sh
root@LAPTOP-BIOOQKCV:~# ./script4.sh /var/log/dpkg.log install
========================================
          Log File Analyzer
========================================
Analyzing File : /var/log/dpkg.log
Keyword        : 'install'
----------------------------------------
Total occurrences of 'install' found: 1843
----------------------------------------
Last 5 matching lines:
2025-01-06 20:15:36 status installed packagekit-tools:amd64 1.2.8-2ubuntu1.1
2025-01-06 20:15:37 status installed software-properties-common:all 0.99.49.1
2025-01-06 20:15:37 status installed ubuntu-wsl:amd64 1.539.1
2025-01-06 20:15:37 status installed libgdk-pixbuf-2.0-0:amd64 2.42.10+dfsg-3ubuntu3.1
2025-01-06 20:15:37 status installed dbus:amd64 1.14.10-4ubuntu4.1
========================================
```

Script 5 — Open Source Manifesto Generator

This script takes input from the user and generates a text file.

This was one of the most interesting scripts because it involved user interaction. Instead of just displaying output, it created something based on the input.

It helped me understand how scripts can be used to create simple but meaningful programs.

```
root@LAPTOP-BIOOQKCV:~# nano script5.sh
root@LAPTOP-BIOOQKCV:~# chmod +x script5.sh
root@LAPTOP-BIOOQKCV:~# ./script5.sh
========================================
    Open Source Manifesto Generator
========================================
Enter a tool you love: Git
What does Open Source mean to you in one word? Freedom
What will you build next? AI Project
----------------------------------------
Manifesto saved to manifesto_root.txt
Reading your manifesto below:
----------------------------------------
--- MY OSS MANIFESTO ---
Developer: Vedant Patil
Date: Sun Mar 29 18:34:51 UTC 2026
-------------------------
I use Git to change the world.
To me, Open Source is all about Freedom.
My next contribution will be: AI Project.
========================================
```

After completing all five scripts, I realized that shell scripting is more about understanding logic than memorizing commands.

Even simple scripts can automate tasks and improve efficiency. Working in the Ubuntu terminal gave me practical exposure to how Linux systems operate.

Real-World Applications of Git

When I first heard about Git, I thought it was mainly used to upload code. But after working on this project, I realized that it is used almost everywhere in software development. One of its most important uses is in team projects where multiple developers are working on the same code. Without Git, it would be very easy for people to overwrite each other's work or lose track of changes. Git solves this problem by allowing everyone to work separately and then combine their changes in an organized way.

Another thing I understood is how Git keeps a record of every change. Instead of replacing files again and again, it stores a complete history. This becomes very useful when something goes wrong, because you can simply go back to an earlier version. Git is also used in automated systems where testing and deployment happen automatically after changes are made. This made me realize that Git is not just a tool for storing code, but part of the entire development process. It can even be used for managing documents or notes where tracking changes is important.

Learning Outcomes

While working on this project, I learned both theoretical and practical concepts. On the theoretical side, I understood why Git was created and what problem it solved. Earlier, I never really thought about the reason behind such tools, but now it makes more sense. I also learned about open-source software and how it works beyond just being free to use.

On the practical side, working with shell scripts helped me become more comfortable with the Linux environment. In the beginning, I was not very confident using the terminal and used to hesitate before typing commands. But after practicing through scripts, I became more comfortable. One important thing I learned is that understanding concepts is more useful than

memorizing commands. I also realized that making small mistakes and fixing them actually helped me learn better. Overall, this project helped me connect theory with practical experience.

Future of Git

From what I observed, Git is not something that will disappear anytime soon. In fact, it seems to be becoming even more important as projects grow larger and more complex. Managing code properly becomes necessary in such cases, and Git already handles this effectively. I also noticed that Git is now closely integrated with platforms like GitHub and GitLab, which provide additional features like automation, testing, and project management.

Git is also becoming an important part of DevOps, where it connects development and deployment processes. At the same time, it can feel a bit difficult for beginners at first, so in the future it may become more user-friendly. However, I don’t think Git will be replaced. It will most likely continue to improve and adapt to new technologies.

Limitations of Git

Even though Git is very useful, it does have some limitations. One of the main challenges is that it can be difficult to learn at the beginning. There are many concepts involved, and it can feel confusing at first. I personally faced some difficulty understanding how things work initially.

Another limitation is that Git works best with text files and is not ideal for large binary files like videos or images. In larger projects, managing multiple branches and resolving conflicts can also become complicated. Since Git is a distributed system, it requires proper discipline from users, otherwise things can become confusing. However, with practice and proper usage, most of these issues can be managed.

Conclusion

After completing this project, I now have a better understanding of how open-source tools work. Earlier, I used tools without thinking much about how they were designed, but now I understand that Git is not just a tool, but a system created to solve real problems in software development.

Even though my practical work mainly focused on shell scripting, it still helped me understand how Linux systems work and how tools operate within them. One important thing I realized is that learning becomes easier when theory and practical work are combined. Overall, this project improved my understanding and gave me more confidence, especially while working with the Linux terminal.